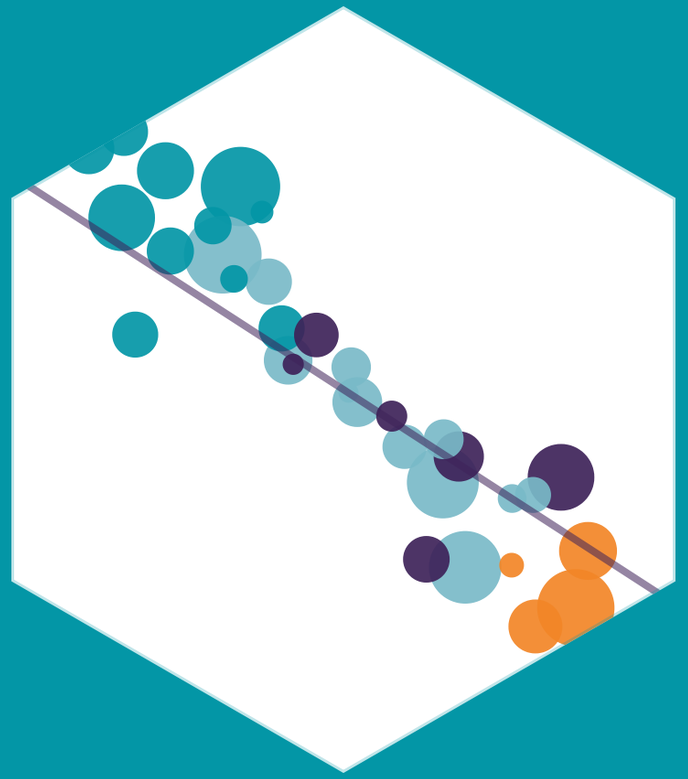
ECONOMETRICS

TA SESSION 5

Barcelona School of Economics
Lucia Sauer
lucia.sauer@bse.eu



Overview

- Monte Carlo experiments: consistency of the sample mean and of OLS
- Heteroskedasticity: visual inspection, Breusch-Pagan test, robust standard errors
- Bootstrapped standard errors: the paired bootstrap

Monte Carlo Experiments: Consistency

From TA3 to consistency

In TA3 we **fixed** n and looked at the sampling distribution of an estimator across many samples. Now we let n **grow**.

Consistency

An estimator $\hat{\theta_n}$ is **consistent** for θ if it converges in probability to θ:

$$\text{plim}_{n\to\infty}\ \hat{\theta_n} = \theta$$

The picture we want to see: as n grows, the sampling distribution of $\hat{\theta_n}$ **collapses** onto the true value θ.

The Monte Carlo recipe is the same as in TA3, repeated for several sample sizes.

Seeing it: the sample mean

1 · POPULATION

Exponential

2 · SAMPLE SIZE

$n = 30$

Sample

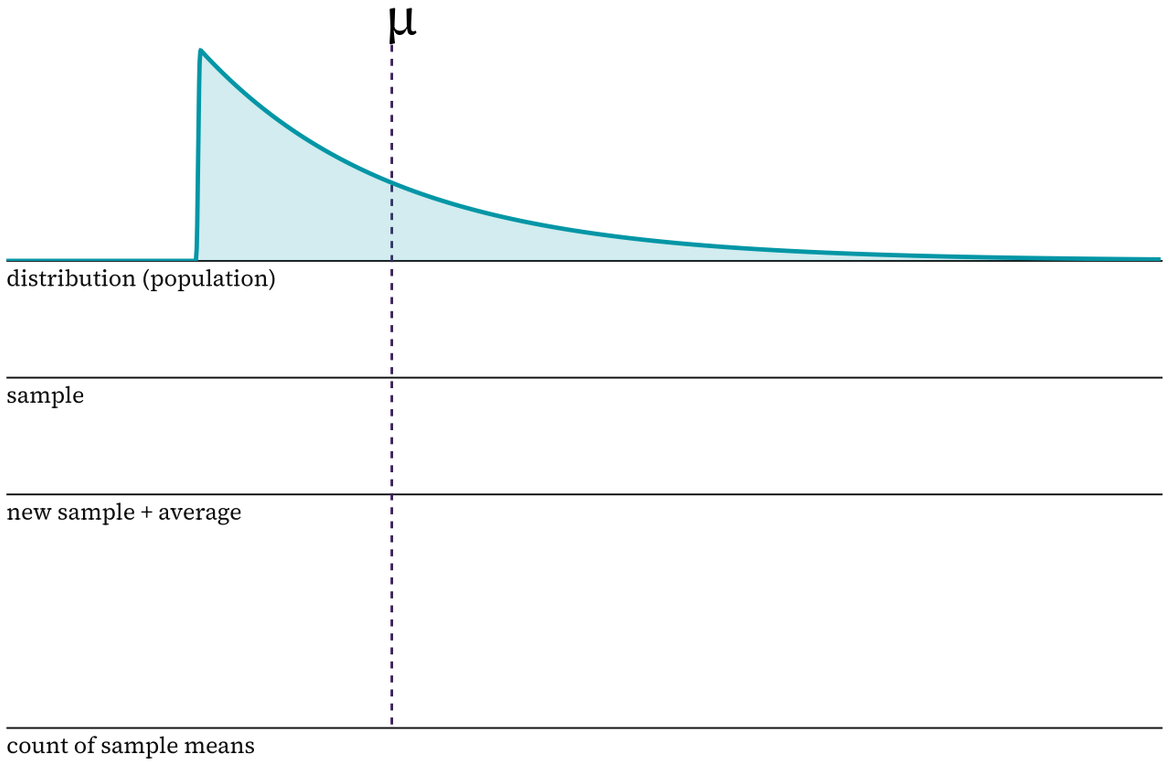
3 · REPEAT THE EXPERIMENT

Draw new samples from the population and average each one.

New sample | 100 new samples

Move n from 10 to 1,000 (click **Sample** after changing it) and watch the histogram collapse onto μ.

Adapted from Seeing Theory (D. Kunin et al., Brown University).



Click **Sample** to draw your one observed sample.

Consistency of the sample mean: your dgp

```python
rng = np.random.default_rng(0)

def simulate_means(n, R=1000, mu=0, sigma=1):
    """Generate R samples of size n de from a Normal(mu, sigma) and return R means"""
    means = np.zeros(R)
    for r in range(R):
        sample = rng.normal(loc=mu, scale=sigma, size=n)   # a sample of size n
        means[r] = sample.mean()                           # store the mean
    return means

# example: z ~ Normal(2, 1)
mu, sigma = 2, 1
ns = [10, 100, 1_000, 10_000]
means = {n: simulate_means(n, mu=mu, sigma=sigma) for n in ns}
```

Your turn

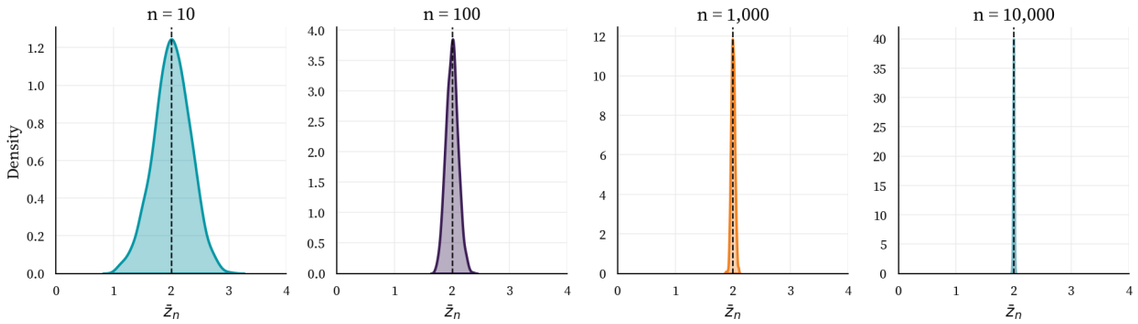
Propose another dgp for z: change `rng.uniform(0, 10, size)`, `rng.chisquare(4, size)`, `rng.poisson(3, size)` and re-run the next plot.

The sample mean collapses onto μ

CODE

```
fig, ax = plt.subplots(figsize=(10, 4.2))
for n in ns:
    sns.kdeplot(means[n], fill=True, alpha=0.3, label=f"n = {n:,}", ax=ax)
ax.axvline(mu, color="black", ls="--", label=r"$\mu$")
ax.set_xlabel(r"$\bar z_n$")
ax.legend()
plt.show()
```

OUTPUT



Same x-axis in every panel: as n grows the KDE gets **narrower and taller** around $\mu = 2$.

Consistency of OLS: a dgp with four regressors

$$y_i = 1 + 2\,x_{i2} - 1\,x_{i3} + 0.5\,x_{i4} + \varepsilon_i$$

$$x_{i2} \sim N(0,1), \qquad x_{i3} = 0.5\,x_{i2} + u_i, \quad u_i \sim N(0,1), \qquad x_{i4} \sim \text{Bernoulli}(0.3)$$

$$\varepsilon_i = \frac{\chi^2_i(3) - 3}{\sqrt{6}} \quad \text{(independent of } X\text{)}$$

- x_2 and x_3 are **correlated**; x_4 is a dummy
- The error is **not normal** (skewed), but it has mean 0 and is independent of the regressors

Recall from lecture

OLS is consistent if the observations are i.i.d., $E(x_i\varepsilon_i) = 0$, and plim $\frac{X'X}{n}$ exists and is finite and non-singular. Normality is **not** needed.

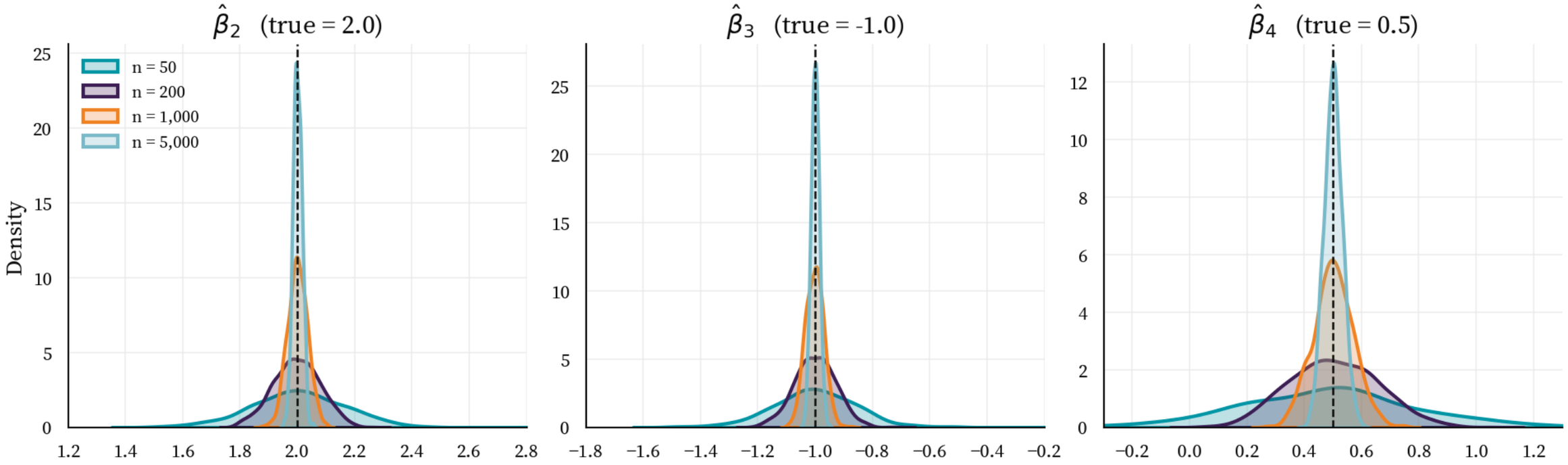
Monte Carlo in code

```
BETA = np.array([1, 2, -1, 0.5])

def simulate_ols(n, R=1000, seed=0):
    rng = np.random.default_rng(seed)
    betas = np.empty((R, 4))
    for r in range(R):
        x2 = rng.normal(0, 1, n)
        x3 = 0.5 * x2 + rng.normal(0, 1, n)
        x4 = rng.binomial(1, 0.3, n)
        eps = (rng.chisquare(3, n) - 3) / np.sqrt(6)
        X = np.column_stack([np.ones(n), x2, x3, x4])
        y = X @ BETA + eps
        betas[r] = np.linalg.solve(X.T @ X, X.T @ y)
    return betas

ns_ols = [50, 200, 1_000, 5_000]
betas_by_n = {n: simulate_ols(n) for n in ns_ols}
```

OLS estimates collapse onto the true β's



For **every** estimated coeff., the KDE gets narrower and taller around the true value as n grows, even though ε is not normal.

Non-spherical disturbances

Spherical disturbances

The **spherical disturbances** assumption includes two assumptions on the behaviour of the disturbances:

1. **Homoskedasticity**: $var(\varepsilon_i \mid X) = \sigma^2$
2. **Uncorrelatedness**: $cov(\varepsilon_i, \varepsilon_j \mid X) = 0, \quad \forall i \neq j$

$$Var(\varepsilon \mid X) = \sigma^2 I_n$$

Now we consider the possibility of abandoning one of these assumptions, and see how estimation and inference change.

1. Heteroskedasticity

Let's start by relaxing the homoskedasticity assumption:

$$var(\varepsilon_i \mid X) = \sigma_i^2$$

The variance-covariance matrix of the disturbances is now:

$$\Sigma = \begin{bmatrix} \sigma_1^2 & 0 & 0 & \cdots & 0 \\ 0 & \sigma_2^2 & 0 & \cdots & 0 \\ 0 & 0 & \sigma_3^2 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & \sigma_n^2 \end{bmatrix}$$

House prices data

The `hprice3` dataset (Wooldridge) contains 321 house sales. We estimate:

$$price_i = \beta_1 + \beta_2\, area_i + \beta_3\, rooms_i + \varepsilon_i$$

```
import wooldridge as woo
import statsmodels.formula.api as smf

df = woo.dataWoo("hprice3")
model = smf.ols("price ~ area + rooms", data=df).fit()
df[["price", "area", "rooms", "nbh"]].head(4)
```

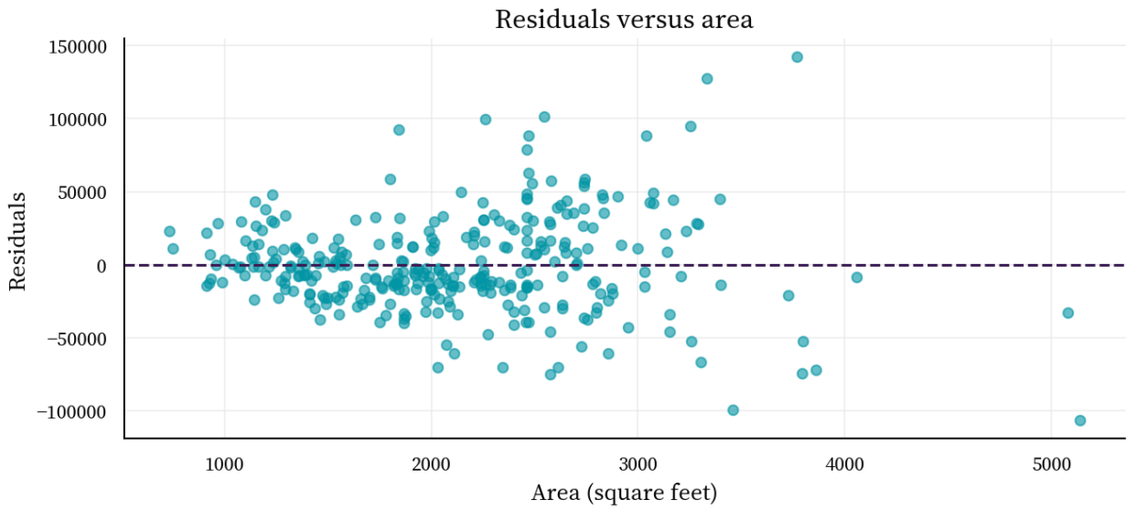
	price	area	rooms	nbh
0	60000.0	1660	7	4
1	40000.0	2612	6	4
2	34000.0	1144	6	4
3	63900.0	1136	5	4

1.1 Visual inspection of heteroskedasticity

CODE

```
fig, ax = plt.subplots(figsize=(9, 4.2))
ax.scatter(df["area"], model.resid, color=bse_style.TEAL, s=30, alpha=0.6)
ax.axhline(0, color=bse_style.PURPLE, ls="--")
ax.set_xlabel("Area (square feet)")
ax.set_ylabel("Residuals")
plt.show()
```

OUTPUT



Question

Does the spread of the residuals look constant as `area` grows?

2. Testing for heteroskedasticity

Breusch-Pagan steps

1. Estimate the original model and obtain the residuals $\hat{\varepsilon_i}$
2. Run the auxiliary regression:

$$\hat{\varepsilon_i^2} = \alpha_1 + \alpha_2\, area_i + \alpha_3\, rooms_i + v_i$$

3. Test $H_0 : \alpha_2 = \alpha_3 = 0$ (homoskedasticity) vs. H_1 : at least one $\alpha_j \neq 0$ (heteroskedasticity)

Finite sample version

$$F = \frac{(\widetilde{RSSE} - \widetilde{SSE})/2}{\widetilde{SSE}/(321-3)} \underset{\widetilde{H}_0}{} F_{2,\,321-3}$$

Asymptotic version

$$n\widetilde{R}^2 \underset{\widetilde{H}_0}{\overset{a}{}} \chi^2(2)$$

Breusch-Pagan test in Python

By hand: the auxiliary regression

```python
df["resid2"] = model.resid ** 2
aux = smf.ols("resid2 ~ area + rooms", data=df).fit()

n = len(df)
print(f"F     = {aux.fvalue:.4f}")
print(f"p     = {aux.f_pvalue:.2e}")
print(f"n R^2 = {n * aux.rsquared:.4f}")
```

```
F     = 35.3295
p     = 1.39e-14
n R^2 = 58.3584
```

Question

At $\alpha = 5\%$, do we reject homoskedasticity? What does that imply for the standard errors of our regression?

Classical SE

```python
summary = model.summary(slim=True)
print(summary)
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                  price   R-squared:                       0.430
Model:                            OLS   Adj. R-squared:                  0.426
No. Observations:                 321   F-statistic:                     120.0
Covariance Type:            nonrobust   Prob (F-statistic):           1.52e-39
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept  -2.231e+04   1.36e+04     -1.637      0.103   -4.91e+04    4497.993
area          35.5631      3.115     11.418      0.000      29.435      41.691
rooms       6604.3052   2401.922      2.750      0.006    1878.639    1.13e+04
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 1.68e+04. This might indicate that there are
```

Question

- How does the behaviour of the OLS estimator change under non-spherical disturbances?
- Which properties of the estimator remain and which ones are lost?

Robust SE

Under heteroskedasticity, $\hat{\beta}$ is still unbiased, but $\hat{\sigma}^2(X'X)^{-1}$ is no longer its variance. The **robust** (White, HC1) estimator is:

$$\widehat{Var}_{HC1}(\hat{\beta}) = \frac{n}{n-K}(X'X)^{-1}\left(\sum_{i=1}^{n}\hat{\varepsilon}_i^2\, x_i x_i'\right)(X'X)^{-1}$$

```
model_robust = smf.ols("price ~ area + rooms", data=df).fit(cov_type="HC1")
print(model_robust.summary(slim=True).tables[1])
```

```
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
Intercept  -2.231e+04   1.21e+04     -1.849      0.064    -4.6e+04    1336.364
area          35.5631      4.817      7.383      0.000      26.122      45.004
rooms       6604.3052   2501.109      2.641      0.008    1702.221    1.15e+04
==============================================================================
```

Question

Which elements of the output change and why?

Bootstrapped Standard Errors

The bootstrap idea

The robust SE needs a **formula**. For many estimators the formula is hard (or impossible) to derive. The **bootstrap** estimates the standard error by **resampling** the data we have.

Remember the Monte Carlo experiment from TA3: draw many samples **from the population**, compute the estimator in each, look at the spread.

The problem: with real data we do not know the population.

> **The bootstrap trick**
>
> Treat the **observed sample** as if it were the population: draw many samples of size n **with replacement** from it, compute the estimator in each, and use the sd of those estimates as the standard error.

Seeing it: Monte Carlo vs. Bootstrap

1 · POPULATION

Exponential

2 · YOUR ONE SAMPLE

$n = 30$

Sample

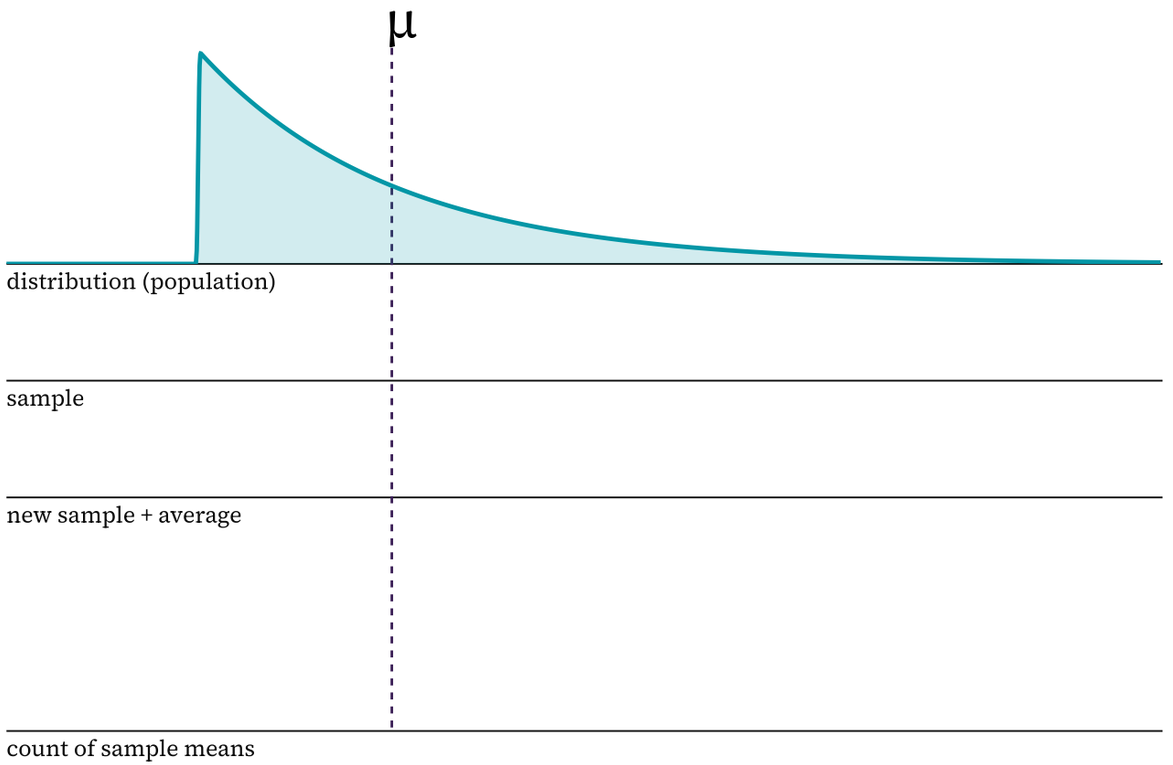
3 · REPEAT THE EXPERIMENT

Monte Carlo | Bootstrap

New sample | 100 new samples

Run both modes: the active one is drawn as bars, the other one as an outline.

Adapted from Seeing Theory (D. Kunin et al., Brown University).



Click **Sample** to draw your one observed sample.

Paired bootstrap SE for OLS

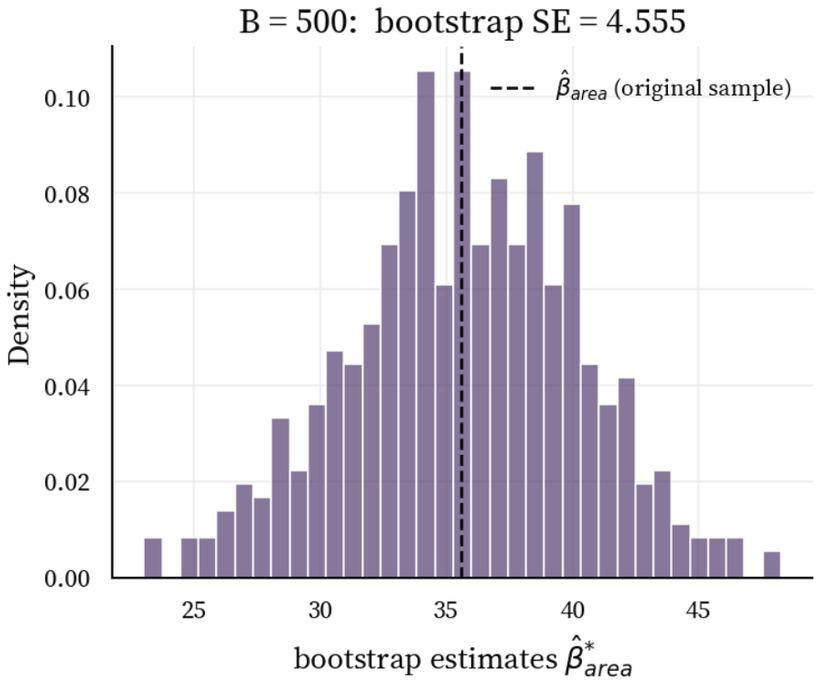
The same recipe works for $\hat{\beta}$: resample **rows** (y_i, x_i) with replacement, keeping each y_i together with its x_i (the **paired** bootstrap).

```python
B = 500
rng = np.random.default_rng(123)

boot = []
for b in range(B):
    df_b = df.sample(n=len(df), replace=True, random_state=rng)
    fit_b = smf.ols("price ~ area + rooms", data=df_b).fit()
    boot.append(fit_b.params)

boot = pd.DataFrame(boot)
boot_se = boot.std()
boot_se
```

```
Intercept    11898.687112
area             4.554624
rooms         2391.799973
dtype: float64
```



Bootstrap regression table

With the bootstrap SE we build the usual table, using normal-based inference:

```
from scipy import stats

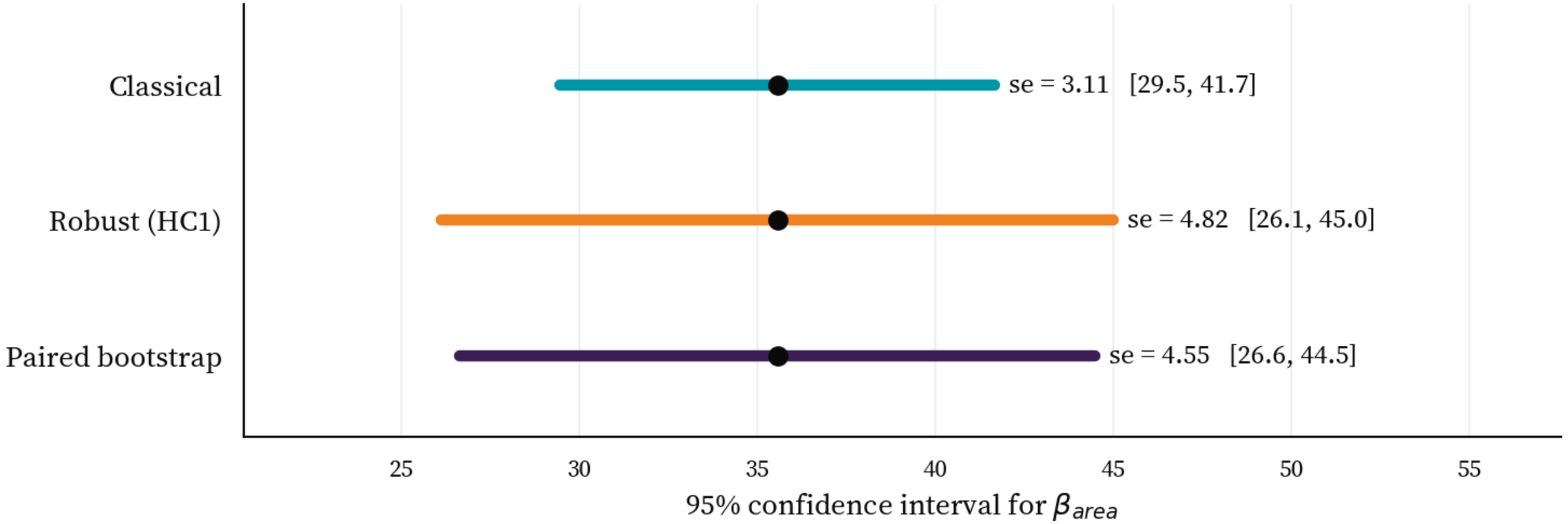
table = pd.DataFrame({"coef": model.params, "boot se": boot_se})
table["z"] = table["coef"] / table["boot se"]
table["P>|z|"] = 2 * (1 - stats.norm.cdf(table["z"].abs()))
table["[0.025"] = table["coef"] - 1.96 * table["boot se"]
table["0.975]"] = table["coef"] + 1.96 * table["boot se"]
table.round(4)
```

	coef	boot se	z	P>|z|	[0.025	0.975]
Intercept	-22314.8520	11898.6871	-1.8754	0.0607	-45636.2788	1006.5747
area	35.5631	4.5546	7.8081	0.0000	26.6360	44.4901
rooms	6604.3052	2391.8000	2.7612	0.0058	1916.3772	11292.2331

Note

Bootstrap SEs depend on the random draws: a different `seed` or a different B gives slightly different numbers (Stata's `bootstrap, reps(500)` will not match exactly).

Comparing confidence intervals across SE types



Same coefficient estimate for `area` *, but wider confidence intervals under robust and paired bootstrap SEs.*

Thanks!